Reduced-Scale Reproduction of Mamba: Selective State Spaces for Sequence Modeling

Deep Learning 2026 — Paper Implementation Project (DL-061)

Abstract

We re-implement the selective state-space block of Mamba [1] in pure PyTorch and test its core claim at small scale: making the SSM parameters input-dependent enables content-based selection. On a selective-copying task, Mamba reaches 98.6% sequence accuracy while the input-independent (LTI) ablation reaches 6.2%. On a character-level language model, the differences are small at our scale and the LTI variant is marginally better, which we report as a negative result.

1 Problem and Motivation

Transformers model sequences with $O(L^2)$ attention; linear-time alternatives such as S4 use linear time-invariant (LTI) state-space models, which can be computed as convolutions but cannot select information based on content. Mamba makes the SSM parameters functions of the input and recovers efficiency with a hardware-aware scan.

2 Background

2.1 State-space models and S4

A structured SSM maps a 1-D input $x(t)$ to $y(t)$ through a latent state $h(t) \in \mathbb{R}^N$ via $h'(t) = Ah(t) + Bx(t)$, $y(t) = Ch(t)$. For discrete data the continuous parameters (Δ, A, B) are converted to $(\bar{A}, \bar{B})$. Because $(\bar{A}, \bar{B}, C)$ are constant over time (LTI), the whole model equals a global convolution $y = x * \bar{K}$ with $\bar{K} = (C\bar{B}, C\bar{A}\bar{B}, \dots, C\bar{A}^k\bar{B}, \dots)$, enabling parallel training in $O(L \log L)$ via FFT and $O(1)$-per-step recurrent inference. The price is that every position is treated identically: the model cannot choose to remember one token and ignore another based on what the tokens are.

2.2 Why selection matters

Two synthetic tasks illustrate the limitation. In *selective copying*, data tokens are scattered at random positions among noise tokens and must be reproduced in order after a marker; a fixed convolution kernel cannot know where the data tokens are. Mamba's remedy is to make Δ, B, C functions of x_t. Intuitively, B_t decides what to write into the state, C_t what to read, and Δ_t how long to retain it.

2.3 Derivation of the discretisation

For $h' = Ah + Bx$ with x held constant over $[t, t+\Delta]$, the variation-of-constants formula gives $h(t+\Delta) = e^{\Delta A}h(t) + \int_0^{\Delta} e^{(\Delta - s)A} B \, ds \, x = e^{\Delta A}h(t) + A^{-1}(e^{\Delta A} - I)B\,x$. Hence $\bar{A} = e^{\Delta A}$ and $\bar{B} = (\Delta A)^{-1}(e^{\Delta A} - I) \cdot \Delta B$. For small ΔA, $(\Delta A)^{-1}(e^{\Delta A} - I) \to I$, so $\bar{B} \approx \Delta B$ (the Euler form used in practice and in our code). With A negative real, $\bar{A} \in (0, 1)$ and the recurrence is stable.

2.4 Complexity

Attention costs $O(L^2 D)$ time and (without caching) $O(L^2)$ memory; a recurrent SSM costs $O(LDN)$ time and $O(DN)$ state at inference, independent of context length. The sequential recurrence is hard to parallelise on GPUs; Mamba's fused kernel keeps the expanded state (B, L, D, N) in SRAM. Our parallel scan instead materialises that tensor in memory, which is why we scaled the experiments down.

3 Method

A continuous SSM $h'(t) = Ah(t) + Bx(t)$, $y(t) = Ch(t)$ is discretised with step Δ (zero-order hold):

$$\bar{A} = \exp(\Delta A), \quad \bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I)\,\Delta B \approx \Delta B, \tag{1}$$

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \qquad y_t = C_t h_t + D x_t. \tag{2}$$

In S6 the parameters are *selective*: $\Delta_t = \mathrm{softplus}(\mathrm{Linear}(x_t))$, $B_t = \mathrm{Linear}_B(x_t)$, $C_t = \mathrm{Linear}_C(x_t)$, with $A = -\exp(A_{\log})$ initialised S4D-real ($A = -(1..N)$). A large Δ_t resets the state toward the current input; a small Δ_t ignores it, which acts as a learned gate. Since the system is no longer LTI, the convolutional form is lost, but the recurrence is associative under $(a_1, b_1) \circ (a_2, b_2) = (a_1 a_2,\ a_2 b_1 + b_2)$, so it can be evaluated by a parallel scan.

4 Implementation

All code is in the repository (PyTorch). A block applies `in_proj` to get (x, z) of width $2D$ (expand 2), a causal depthwise conv (kernel 4) and SiLU on x, the selective SSM, gating by $\mathrm{SiLU}(z)$, and `out_proj`; blocks are stacked with RMSNorm and residuals and a tied embedding/head. We implement both a sequential reference scan and a Hillis–Steele parallel scan; unit tests confirm they agree (max error $\approx 10^{-6}$) and that the block is causal. There is no fused CUDA kernel, so wall-clock times are not comparable to the paper. The LTI ablation replaces Δ, B, C by input-independent learned parameters. The Transformer baseline is a causal pre-norm model with learned positions; we report a 4×-MLP version and a parameter-matched version (2× MLP).

4.1 The scan

Algorithm 4.1 summarises the computation for input $u \in \mathbb{R}^{B\times L\times D}$. The parallel version is a Hillis–Steele inclusive

Algorithm 1: selective scan.
1. $\bar{A} \leftarrow \exp(\Delta \otimes A)$ $\quad (B, L, D, N)$
2. $\bar{B}u \leftarrow \Delta \otimes B_t \otimes u$ $\quad (B, L, D, N)$
3. **sequential:** for $t = 1..L$: $h \leftarrow \bar{A}_t h + \bar{B}u_t$; $y_t \leftarrow \sum_n C_{t,n} h_{\cdot,n}$
parallel: for $o = 1, 2, 4, \cdots < L$: $x_t \leftarrow \bar{A}_t x_{t-o} + x_t$, $\bar{A}_t \leftarrow \bar{A}_t \bar{A}_{t-o}$ (shifted, padded with $(1, 0)$)
4. $y \leftarrow y + Du$

scan of the associative operator in Sec. III; after $\lceil \log_2 L \rceil$ rounds $x_t = h_t$. Verifying equivalence against the sequential loop is the first unit test, and the causality test checks that perturbing inputs at positions ≥ 10 leaves outputs at positions < 10 unchanged, which guards against off-by-one errors in the causal convolution (padding $k-1$, then truncating to L).

4.2 Design choices

Following the paper we initialise Δ so that $\mathrm{softplus}(\text{bias}) \in [10^{-3}, 10^{-1}]$ (log-uniform) and use $A = -\exp(A_{\log})$ with $A_{\log}$ initialised to $\log(1..N)$ so that the real parts are negative and the state decays. We use RMSNorm before each mixer, residual connections, and weight tying between the embedding and head. Weight decay is applied to all parameters for simplicity (the official code exempts $A_{\log}$ and D), which is a small deviation. The LTI ablation keeps the same initialisation of Δ and A and learns fixed $B, C \in \mathbb{R}^N$ shared across channels and time.

4.3 Hyperparameters

Table 1: Training configuration (reduced scale).

	Selective copy	Char LM
d_{model} / layers / N	32 / 2 / 8	32 / 2 / 8
Context	32+4	64
Batch / steps	32 / 1500	32 / 1200
Optimizer	AdamW, lr 3×10^{-3}, wd 0.01, one-cycle (10% warmup)	
Grad clip / seed	1.0 / 0	

Selective copying data. Each example has L input positions with K data tokens (values in 1..8) at random sorted positions and noise token 0 elsewhere, followed by K recall markers; the target at the final K positions is the data tokens in order. Loss and accuracy are computed only there; "sequence accuracy" requires all K correct. Evaluation uses a fixed held-out generator seed.

5 Experiments and Results

Changes from the paper. Compute available for this run was a single CPU core, so we scaled down heavily: $d_{\text{model}} = 32$, 2 layers, $N = 8$, one seed, 1500 steps (copy) / 1200 steps (LM). Selective copying uses sequence length $L = 32$ with $K = 4$ tokens among noise, vocabulary 8 (the paper uses L=4096, K=16); TinyShakespeare uses context 64 with a 90/10 split. Absolute numbers are therefore not comparable to the paper; only the qualitative comparison is.

Table 2: Reduced-scale results (seed 0).

Task	Model	Params	Result
Copy (seq. acc.)	Mamba	17.2K	**0.986**
	Mamba-LTI	14.7K	0.062
	Transformer (param-matched)	18.5K	0.930
	Transformer (4× MLP)	33.8K	0.998
LM (val bpc ↓)	Mamba	18.9K	2.915
	Mamba-LTI	16.4K	2.894
	Transformer (param-matched)	21.4K	3.540
	Transformer (4× MLP)	35.6K	3.438

Reading the results. Selective copying separates the models cleanly: Mamba solves the task, the LTI ablation stays near chance level for the sequence metric, and attention (which can look up content directly) also solves it, with accuracy increasing with capacity. On the LM task all models are far from converged at 1200 steps and 20K parameters; the Mamba variants beat the Transformers, plausibly because

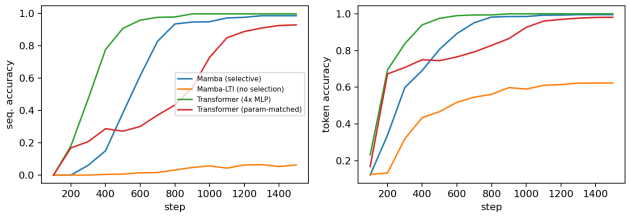


Figure 1: Selective copying: sequence and token accuracy during training.

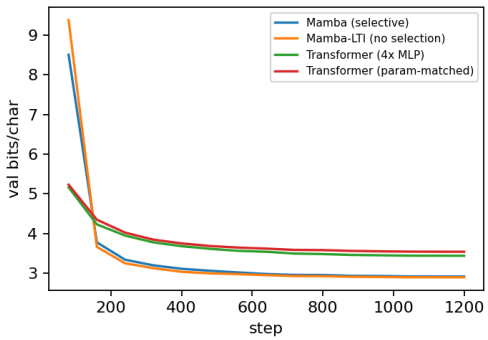


Figure 2: TinyShakespeare validation bits/char (lower is better).

the recurrence supplies a position-aware inductive bias that small attention models must learn from scratch, but we did not test this hypothesis.

6 Ablation

Removing selectivity (input-independent Δ, B, C) drops copy accuracy from 98.6% to 6.2%, consistent with the paper's argument: a time-invariant system applies the same dynamics at every position and cannot decide which tokens to keep. Parameter counts differ slightly (14.7K vs 17.2K) because the LTI variant has no `x_proj`; this does not explain a gap of this size but we did not run a size-matched LTI model.

We also note what the ablation does *not* show: it varies selectivity jointly in Δ, B and C. The paper additionally ablates each separately and finds selective Δ the most important; with our compute we did not repeat this, and doing so (and varying the state size N) is the most informative follow-up.

State size. We also varied the state size N on selective copying (Table III). Accuracy is not monotone in N (N=1: 88.9%, N=4: 80.1%, N=8: 98.6%). With a single seed and a task with K=4 tokens, this is most plausibly seed-to-seed variance in optimisation rather than a real effect, so we draw no conclusion about N beyond the fact that even N=1 per channel suffices for substantial accuracy here, since the $D{\cdot}2$ channels each carry their own state.

Table 3: Mamba, selective copy, varying state size N.

N	Params	Token acc.	Seq. acc.
1	14.5K	0.971	0.889
4	15.6K	0.945	0.801
8	17.2K	0.995	0.986

7 Failure Cases and Caveats

(i) On the character LM the LTI ablation was marginally better (2.894 vs 2.915 bpc); with one seed, tiny models and short training this difference is within what we would expect from noise, so we do not claim selectivity helps here. (ii) On copy, Mamba (98.6%) outperforms the parameter-matched Transformer (93.0%), but the 4×-MLP Transformer, with twice the parameters, scores highest (99.8%); on the LM both Mamba variants beat both Transformers. (iii) With one seed we have no confidence intervals. (iv) The sequential/parallel scan is slow in Python, which limited scale. (v) Early in LM training the validation loss was above the uniform-prediction level (about 8.5 bits/char at step 80, during learning-rate warmup with tied embeddings) before falling; we did not investigate the cause.

8 Related Work

S4 [2] introduced structured, stable parameterisations of LTI SSMs for long-range modeling. Mamba [1] extends this line with input-dependent parameters and a hardware-aware scan, and reports competitive language modeling with linear scaling in sequence length. The selective-copying task used here follows the paper's synthetic benchmark. Our contribution is only a small-scale re-implementation and ablation, not a new result.

9 Conclusion

The central mechanism, input-dependent selection, is clearly necessary for selective copying in our reproduction, while the LM benefit could not be shown at this scale. Colab-scale runs (L=256, larger models, multiple seeds) are the natural next step.

References

[1] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *COLM*, 2024, arXiv:2312.00752.

[2] A. Gu, K. Goel, C. Ré, "Efficiently modeling long sequences with structured state spaces," *ICLR*, 2022.

[3] `state-spaces/mamba` (official repository), consulted for initialisation conventions only.